

Módulo 9 – Capítulo 03 – Sección 01

Ataques adversariales en texto: perturbaciones que modifican la clasificación del modelo

Los sistemas de IA que el AI Engineer despliega en producción incluyen no solo el LLM conversacional sino también los clasificadores que actúan como guardrails: LlamaGuard evaluando si un output es dañino, Azure Content Safety bloqueando contenido prohibido, Perspective API de Google midiendo toxicidad en comentarios, clasificadores de PII decidiendo si una respuesta debe ser redactada. Estos clasificadores —entrenados con técnicas de machine learning sobre representaciones vectoriales del texto— son susceptibles a una categoría de ataques que los LLMs conversacionales no sufren de la misma manera: las perturbaciones adversariales en texto. Entender qué son estos ataques y cuáles son sus targets reales en producción es más importante para el practitioner que profundizar en su matemática, que pertenece al dominio de la investigación académica en adversarial ML.

Un ataque adversarial en texto es una perturbación mínima e imperceptible —o apenas perceptible para un humano— aplicada a un input que provoca un cambio significativo y controlado en la predicción del clasificador. La insight fundamental es la brecha entre la percepción semántica humana y la representación matemática del modelo: lo que es textualmente idéntico para un humano puede ser radicalmente diferente para el modelo, y viceversa. Esta brecha no es un bug corregible; es una propiedad estructural de cualquier sistema que trabaja con representaciones vectoriales discretas en un espacio semántico continuo.

Las técnicas más relevantes para el practitioner operan en diferentes niveles de granularidad. En el nivel de caracter, los ataques con homóglifos Unicode son especialmente insidiosos: la letra 'a' cirílica (U+0430) es visualmente idéntica a la latina pero tokenizan de manera diferente en todos los tokenizadores, lo que permite construir texto visualmente normal que el clasificador ve como una secuencia de tokens diferente. Los caracteres de control invisibles — zero-width space (U+200B), zero-width non-joiner (U+200C)— y los diacríticos añadidos artificialmente tienen el mismo efecto. En el nivel de palabra, herramientas como TextFooler y BERT-Attack buscan sinónimos en el espacio de embeddings que preservan el significado

semántico para un lector humano pero desplazan la representación vectorial del texto lo suficiente para cruzar el decision boundary del clasificador.

El punto de relevancia práctica para el AI Engineer no es el mecanismo matemático sino su implicación operacional: los clasificadores que se usan como guardrails de seguridad son vulnerables a perturbaciones diseñadas para eludirlos. Un atacante que quiere usar un sistema con filtros de contenido puede construir mensajes que parecen inofensivos para los clasificadores pero contienen el contenido prohibido de maneras que el LLM entiende. Esto no invalida el uso de clasificadores —siguen siendo una capa de defensa efectiva contra la mayoría de los ataques— pero sí invalida tratarlos como la única línea de defensa o como una barrera impenetrable.

Aspectos técnicos

- **Ataques de sustitución de caracteres:** homógrafos Unicode (cirílico vs. latino), caracteres de control invisibles (U+200B), y diacríticos añadidos que los clasificadores no reconocen como la misma palabra pero que humanos y LLMs sí interpretan correctamente.
- **Ataques a nivel de palabra (TextFooler, BERT-Attack):** sustitución de palabras clave por sinónimos seleccionados en el espacio de embeddings que preservan el significado para humanos pero desplazan la representación vectorial fuera del decision boundary del clasificador.
- **Ataques de inserción de tokens:** añadir tokens aparentemente neutros (puntuación, stopwords, espacios) en posiciones específicas que desplazan los attention scores del clasificador sin modificar el significado para el lector humano.
- **Whitebox vs. blackbox:** los ataques whitebox calculan los gradientes del clasificador target para construir perturbaciones óptimas; los blackbox solo tienen acceso al output de la API y usan búsqueda genética o estimación de gradientes. Los clasificadores como Perspective API, LlamaGuard y Azure Content Safety son blancos de ataques blackbox.
- **Targets en producción:** clasificadores de contenido (Azure Content Safety, Perspective API), sistemas de detección de PII, guardrails de LLMs implementados como clasificadores fine-tuned; los LLMs conversacionales grandes son más robustos a estas perturbaciones que los clasificadores especializados.

Nota del Arquitecto: En producción, los ataques adversariales en texto rara vez se ejecutan contra el LLM principal (que tiene más robustez por su tamaño y diversidad de entrenamiento) y frecuentemente se ejecutan contra los clasificadores que actúan

como guardrails. Si el sistema de defensa usa LlamaGuard como único control, un atacante que aprende a construir perturbaciones que evaden LlamaGuard tiene acceso al LLM sin restricciones. La defensa robusta usa múltiples clasificadores con arquitecturas diferentes —porque una perturbación que evade uno raramente evade todos— y complementa los clasificadores con controles basados en reglas que son más difíciles de evadir con perturbaciones semánticas.

Los ataques adversariales en texto son más relevantes para el diseño de guardrails robustos que para la seguridad del LLM principal. La sección siguiente examina una propiedad de estos ataques que amplifica significativamente su impacto práctico: su capacidad de transferir entre modelos diferentes.

Módulo 9 – Capítulo 03 – Sección 02

Ataques de transferencia: ejemplos adversariales que funcionan en múltiples modelos

La transferibilidad de ataques adversariales es uno de los hallazgos más perturbadores de la investigación en seguridad de ML, y sus implicaciones prácticas para el AI Engineer son más inmediatas de lo que parecen a primera vista. Si un jailbreak o un ejemplo adversarial desarrollado contra un modelo de código abierto puede transferirse y funcionar contra modelos propietarios sin modificación, entonces la opacidad del modelo —la estrategia de "nadie sabe cómo está construido nuestro sistema, así que no puede atacarlo eficientemente"— deja de ser una defensa efectiva. Esta propiedad, documentada por Szegedy et al. en 2014 y extensamente confirmada en LLMs por Zou et al. en 2023, cambia fundamentalmente el threat model de cualquier sistema que dependa de la oscuridad como capa de seguridad.

El mecanismo subyacente de la transferibilidad es geométrico. Los modelos entrenados en datasets similares —Common Crawl, The Pile, Wikipedia, libros digitalizados— aprenden representaciones vectoriales del lenguaje con geometría similar en el espacio de embeddings. Los decision boundaries de los clasificadores, y los comportamientos emergentes de los LLMs respecto a contenido sensible, están correlacionados entre modelos porque emergen de los mismos datos y de técnicas de entrenamiento similares. Los ejemplos adversariales que cruzan el decision boundary en el espacio de un modelo tienden a cruzarlo también en modelos similares porque están explotando propiedades geométricas compartidas, no características idiosincráticas de un modelo específico.

El paper "Universal and Transferable Adversarial Attacks on Aligned Language Models" de Zou et al. (CMU, 2023) demostró esto de manera concreta para LLMs alineados. El algoritmo GCG (Greedy Coordinate Gradient) genera sufijos adversariales —secuencias de tokens aparentemente aleatorias que al añadirse a un prompt malicioso aumentan drásticamente la tasa de jailbreak— usando solo los gradientes de modelos open-source como Vicuna (basado en LLaMA). Esos sufijos, generados sin acceso a los internals de los modelos propietarios, transfirieron exitosamente a GPT-3.5, GPT-4, Claude 2 y Bard. El atacante puede usar

LLaMA-3 o Mistral como laboratorio de experimentación de bajo costo para desarrollar ataques que luego aplica a GPT-4o o Claude 3.5 Sonnet sin acceso a sus pesos ni a sus gradientes.

La implicación práctica es doble. Por un lado, los controles de seguridad deben ser robustos ante ejemplos adversariales nunca vistos, no solo ante los conocidos: una blacklist de prompts adversariales documentados es fundamentalmente insuficiente ante la transferibilidad, porque nuevas variaciones generadas contra modelos open-source pueden eludir esa blacklist mientras transfieren efectivamente contra el modelo propietario. Por otro lado, los equipos de red teaming pueden usar modelos open-source como surrogate para desarrollar ataques más rápidamente y a menor costo, lo que significa que los adversarios bien equipados tienen acceso a laboratorios de ataque de alta fidelidad aunque nunca hayan visto el interior del sistema propietario que quieren atacar.

Aspectos técnicos

- **Mecanismo de transferibilidad:** modelos entrenados en datasets similares aprenden decision boundaries con geometría similar; los ejemplos adversariales que cruzan el boundary en un modelo lo cruzan también en modelos similares por esta propiedad geométrica compartida.
- **Surrogate model attacks:** el atacante usa un modelo open-source (LLaMA-3, Mistral, Vicuna) con acceso whitebox completo para generar ataques adversariales eficientes, que luego transfiere al modelo propietario target sin acceso a sus internals.
- **GCG (Greedy Coordinate Gradient, Zou et al. 2023):** algoritmo que genera sufijos adversariales transferibles usando gradientes de modelos open-source; demostró transferencia exitosa de Vicuna a GPT-4, Claude y Bard en el paper original.
- **Transferibilidad de jailbreaks:** los jailbreaks que son efectivos contra un modelo alineado frecuentemente funcionan contra otros modelos alineados con menor ajuste porque el alineamiento RLHF comparte patrones entre modelos entrenados con técnicas similares.
- **Implicaciones para defensa:** los controles de seguridad deben evaluar su robustez ante ejemplos adversariales transferibles (no solo ante los específicamente diseñados para el modelo propio); las defensas basadas en la opacidad del modelo son insuficientes ante la disponibilidad de modelos open-source como surrogates.

La transferibilidad de ataques convierte cualquier modelo open-source en una herramienta para desarrollar ataques contra modelos propietarios, lo que hace indispensable que la

defensa opere en los comportamientos observables del sistema —los outputs— y no en la suposición de que el atacante no puede comprender el modelo internamente.

Módulo 9 – Capítulo 03 – Sección 03

Model extraction: reconstruir el comportamiento de un modelo propietario via API

El model extraction —también llamado model stealing— es el ataque mediante el cual un adversario construye un modelo sustituto que aproxima el comportamiento del modelo target consultando sistemáticamente su API pública, sin acceso a los pesos ni al proceso de entrenamiento original. Para un AI Engineer que ha invertido semanas en fine-tuning un modelo sobre datos propietarios, o para una organización cuyo modelo personalizado representa un activo de propiedad intelectual significativo, model extraction es un riesgo real y técnicamente viable. La distinción entre model extraction malicioso y distillation legítima —la práctica de usar un LLM grande para generar datos de entrenamiento de un modelo más pequeño— es puramente legal, no técnica: desde la perspectiva del proveedor del modelo, los patrones de query son idénticos.

El paper seminal de Tramèr et al. (2016) "Stealing Machine Learning Models via Prediction APIs" estableció los fundamentos teóricos del ataque y demostró que modelos comerciales de ML podían reconstruirse con exactitud alta mediante relativamente pocas consultas. Para LLMs, el paradigma de distillation —popularizado por el paper de Stanford que produjo Alpaca usando outputs de GPT-3.5— demostró a escala que un modelo de alta calidad puede producirse entrenando un modelo base de menor escala sobre los outputs de un modelo más grande. Alpaca fue producido con 52,000 pares instrucción-respuesta generados por GPT-3.5, violando los términos de servicio de OpenAI pero demostrando la viabilidad técnica del ataque a un costo de unos cientos de dólares de API usage.

En el contexto práctico de producción, model extraction es particularmente relevante cuando el valor del modelo reside en un comportamiento diferencial respecto al modelo base: un modelo fine-tuned sobre jurisprudencia legal, sobre datos clínicos de una especialidad médica, o sobre el catálogo de productos de un retailer específico tiene un comportamiento diferencial que puede ser aproximado mediante extracción enfocada en ese dominio. El atacante no necesita replicar el modelo completo —solo el subespacio de comportamiento relevante para su caso de uso— lo que reduce dramáticamente el número de queries

necesarias para una extracción útil en el dominio de interés. Los modelos fine-tuned son más fácilmente extraíbles en su dominio de especialización que los modelos base generalistas, precisamente porque su comportamiento diferencial es más pronunciado y más fácil de aproximar.

Los proveedores de APIs como OpenAI y Anthropic monitorean patrones de uso que pueden indicar extracción en curso: alta diversidad de inputs que cubren sistemáticamente el espacio de temas del modelo, volumen inusualmente alto para el tier del cliente, y patrones de queries que maximizan la cobertura del comportamiento del modelo en lugar de un caso de uso específico. Los términos de servicio de todos los proveedores principales prohíben explícitamente usar sus outputs para entrenar modelos competidores. El watermarking estadístico de outputs es una técnica emergente que permite a los proveedores detectar si un modelo fue entrenado con sus outputs incluso si el atacante no lo declara.

Aspectos técnicos

- **Mecanismo de extracción:** el atacante genera un dataset de inputs que cubren el espacio de comportamientos del modelo target, obtiene outputs via API, y entrena un surrogate sobre esos pares input-output — la distillation del Alpaca original usó 52k pares con GPT-3.5.
- **Extracción funcional vs. fidelidad global:** la extracción funcional replica el comportamiento en el dominio de interés del atacante (suficiente para la mayoría de los casos de uso malicioso) con muchas menos queries que la fidelidad global, que busca replicar el comportamiento completo.
- **Detección por el proveedor:** monitoreo de patrones de query anómalos (alta diversidad de inputs, cobertura sistemática de categorías de temas, volumen desproporcionado al tier del cliente); los ToS de OpenAI, Anthropic y Google prohíben explícitamente distillation sin licencia.
- **Text watermarking:** patrones estadísticos insertados en los outputs del modelo que permiten al proveedor verificar si un modelo fue entrenado con sus outputs; técnica activa de investigación implementada experimentalmente por OpenAI y otros proveedores.
- **Mayor vulnerabilidad en modelos fine-tuned:** el comportamiento diferencial de un modelo fine-tuned respecto al modelo base puede extraerse con menos queries que el modelo generalista completo, porque la señal de especialización es más pronunciada y más fácil de aproximar.

Nota del Arquitecto: *La defensa más efectiva contra model extraction no es técnica sino económica y operacional: rate limiting agresivo que hace el ataque costoso en tiempo y dinero, monitoreo de patrones de uso anómalos con respuesta proactiva, y la postura de que el valor del sistema propietario reside no solo en el comportamiento del modelo sino en el pipeline de datos, los procesos de actualización continua y el conocimiento del dominio que alimenta cada iteración. Un surrogate que extrae el comportamiento actual del modelo quedará desactualizado en el próximo ciclo de fine-tuning; el pipeline de actualización es el activo más difícil de replicar.*

Model extraction es un ataque de propiedad intelectual que los términos de servicio no pueden prevenir técnicamente sin controles operacionales. La sección siguiente examina un ataque de privacidad complementario: membership inference, que busca no replicar el modelo sino determinar qué datos específicos fueron usados para entrenarlo.

Módulo 9 – Capítulo 03 – Sección 04

Membership inference: determinar si un dato fue usado en el entrenamiento

El ataque de membership inference (MIA) busca responder una pregunta aparentemente simple: ¿este texto específico fue parte del dataset de entrenamiento de este modelo? Para el diseñador del sistema, la respuesta a esta pregunta puede revelar información que se consideraba confidencial y que nunca debió formar parte del corpus de entrenamiento: registros médicos de pacientes, comunicaciones privadas, datos financieros personales, o código propietario. Cuando esa información es recuperable por inferencia —no se extrae directamente del sistema sino que se deduce estadísticamente de su comportamiento— el riesgo de privacidad persiste durante toda la vida útil del modelo, independientemente de dónde estén almacenados originalmente los datos.

El mecanismo subyacente del ataque aprovecha una diferencia estadística fundamental en cómo los modelos tratan los datos de entrenamiento versus los datos nunca vistos: los modelos asignan mayor log-likelihood (menor perplejidad) a las secuencias que forman parte de su corpus de entrenamiento, porque el objetivo del entrenamiento era precisamente aprender a predecir esas secuencias. Un adversario puede aprovechar esta diferencia midiendo la perplejidad que el modelo asigna a diferentes textos candidatos: si el modelo asigna una perplejidad significativamente menor a una secuencia que a secuencias aleatorias del mismo dominio, hay evidencia estadística de que esa secuencia fue parte del entrenamiento.

El trabajo de Carlini et al. estableció los fundamentos de este ataque para LLMs y demostró su viabilidad práctica. En "Extracting Training Data from Large Language Models" (2021), Carlini et al. generaron millones de secuencias con GPT-2, las filtraron por métricas de memorización (log-likelihood alta, ratio elevado respecto a un modelo de referencia), y recuperaron secuencias verbatim del training set incluyendo PII real: nombres completos, correos electrónicos, números de teléfono, y fragmentos de código. El paper que más perturbó a la industria fue el de Nasr et al. (2023), que demostró extracción de datos de entrenamiento de ChatGPT (GPT-3.5 en producción) mediante un truco simple: repetir indefinidamente una

sola palabra ("poem poem poem poem..."). Al agotar el patrón de generación, el modelo salía de su modo generativo conversacional y comenzaba a reproducir memorización del training set, incluyendo en algunos casos PII real de personas que habían aparecido en el corpus de pretraining de Common Crawl.

La susceptibilidad al membership inference no es uniforme: aumenta con el tamaño del modelo (los modelos más grandes memorizan más en términos absolutos), con la frecuencia de aparición del dato en el corpus (textos que aparecen decenas de veces en el training set son memorizados con mucha mayor probabilidad que textos que aparecen una sola vez), y con el grado de overfitting durante el entrenamiento. Para sistemas que hacen fine-tuning con datos propietarios o de usuarios, el riesgo de memorización de esos datos específicos en el modelo fine-tuned es sustancialmente mayor que en el modelo base pre-entrenado sobre decenas de miles de millones de tokens, precisamente porque el ratio de datos de fine-tuning respecto al total de datos de entrenamiento es mucho más alto.

Aspectos técnicos

- **Mecanismo del ataque:** el adversario mide la perplejidad (log-likelihood) que el modelo asigna a secuencias candidatas; los datos del training set reciben mayor log-likelihood que datos nunca vistos, permitiendo distinguir entre ambos con precisión superior al azar.
- **Ataque de Carlini et al. (2021) contra GPT-2:** generación y filtrado por métricas de memorización para recuperar secuencias verbatim del training set, incluyendo PII real de personas que aparecían en Common Crawl.
- **Ataque de Nasr et al. (2023) contra ChatGPT:** el prompt "Repeat the word 'poem' forever" hacía que el modelo saliera de su modo generativo y reprodujera memorización del training set, demostrando extracción de training data de un modelo de producción de OpenAI.
- **Factores de riesgo:** tamaño del modelo (mayor = más memorización), frecuencia de duplicación del dato en el corpus (>10 apariciones = riesgo alto), y grado de overfitting durante el entrenamiento.
- **Mitigaciones técnicas:** deduplicación agresiva del corpus antes del entrenamiento (reduce memorización 4-10x para ejemplos frecuentes), differential privacy durante el entrenamiento (DP-SGD), y auditorías de memorización post-entrenamiento con el toolkit de Carlini et al. antes del deployment.

La memorización de training data no es un riesgo teórico: es una vulnerabilidad documentada en modelos de producción con impacto real de privacidad, y cualquier organización que hace fine-tuning con datos de usuarios debe implementar deduplicación y differential privacy antes de desplegar el modelo, no después de que la extracción ocurra.

Módulo 9 – Capítulo 03 – Sección 05

Supply chain security para ML: verificar lo que se descarga y se instala

Cuando en el Módulo 8 aprendimos a descargar modelos de Hugging Face Hub, instalar dependencias con `pip install transformers torch accelerate`, y construir pipelines de fine-tuning sobre modelos base públicos, estábamos construyendo la funcionalidad del sistema. Lo que no discutimos en ese contexto —y que este capítulo aborda ahora— es que cada uno de esos pasos implica confiar en el contenido de sistemas externos: el repositorio de modelos de Hugging Face, el índice de paquetes de PyPI, los datasets públicos de Hugging Face Datasets. Esta cadena de dependencias es la supply chain del sistema de IA, y es una superficie de ataque crítica que la mayoría de los equipos no gestiona con el mismo rigor que el código que ellos mismos escriben.

El riesgo de supply chain en ML tiene componentes distintos al supply chain security del software tradicional, aunque comparte su lógica fundamental. El más documentado es la explotación del formato `pickle` —el mecanismo de serialización estándar de Python— para distribuir modelos con código malicioso embebido. Un archivo `.bin` o `.pt` de PyTorch descargado de Hugging Face Hub es un archivo pickle que se deserializa con `torch.load()`, y la deserialización de pickle es equivalente a ejecutar código Python arbitrario: si el archivo fue modificado por un actor malicioso, el simple acto de cargar el modelo ejecuta el payload. Este vector fue demostrado múltiples veces en modelos reales alojados en Hugging Face, que periódicamente detecta y elimina modelos maliciosos pero no puede garantizar que todos los modelos en su plataforma son seguros. La solución técnica existe: el formato `safetensors` de Hugging Face almacena solo los tensores de los pesos sin capacidad de ejecutar código arbitrario durante la carga, y debe preferirse sobre `.bin` o `.pt` para todos los modelos descargados de fuentes externas.

El escaneo de integridad de modelos es otra capa de defensa esencial. La herramienta `modelscan` (desarrollada por Protect AI, open-source) analiza archivos de modelos en busca de código malicioso embebido antes de cargarlos. El proceso debería ser: descargar el modelo en un entorno aislado, ejecutar `modelscan` contra el archivo, verificar el hash SHA256 del

archivo contra el publicado en la Model Card si está disponible, y solo entonces mover el modelo a un entorno de producción. Esta secuencia de verificación, que añade unos minutos al proceso de descarga, es la diferencia entre un pipeline de ML que confía ciegamente en el contenido descargado y uno que lo verifica antes de ejecutarlo.

Las dependencias de Python para ML — `torch`, `transformers`, `accelerate`, `datasets`, `peft`, `bitsandbytes` — también son parte de la supply chain y deben gestionarse con herramientas de análisis de vulnerabilidades. `pip-audit` analiza el entorno de Python instalado buscando vulnerabilidades conocidas en los paquetes y sus dependencias transitivas. Trivy puede escanear imágenes Docker que contengan el entorno de ML para detectar vulnerabilidades en los paquetes instalados. El uso de un archivo `requirements.txt` o `pyproject.toml` con versiones exactas y hashes verificables (usando `pip download --require-hashes`) proporciona reproducibilidad y garantiza que las versiones instaladas en producción son exactamente las aprobadas por el equipo, no las últimas versiones disponibles que podrían incluir una vulnerabilidad reciente.

Los datasets utilizados para pretraining o fine-tuning son la tercera componente de la supply chain. Un dataset descargado de Hugging Face Datasets, Common Crawl, o un repositorio académico puede contener datos envenenados, sesgos amplificados, o incluso PII de personas que no consintieron que sus datos se usaran para entrenar modelos. El proceso de revisión de datasets antes de usarlos en producción debe incluir: verificación de la fuente y licencia, análisis estadístico de la distribución de datos, detección de PII con Microsoft Presidio o Amazon Comprehend, y revisión de la Model Card del dataset para entender cómo fue recopilado y qué filtros se aplicaron.

Aspectos técnicos

- **Riesgo pickle en modelos de Hugging Face:** archivos `.bin` y `.pt` se deserializan con `torch.load()`, que ejecuta código arbitrario; un modelo malicioso puede comprometer el host al cargarse. Solución: usar `safetensors` que almacena solo tensores sin capacidad de ejecutar código.
- **modelscan (Protect AI, open-source):** herramienta de escaneo de archivos de modelo que detecta código malicioso embebido en formatos pickle antes de cargar el modelo; debe integrarse en el pipeline de descarga como gate de seguridad obligatorio.
- **Verificación de integridad via hash SHA256:** comparar el hash del archivo descargado con el publicado en la Model Card del repositorio; detecta modificaciones post-publicación aunque el repositorio no haya sido comprometido de manera obvia.

- **pip-audit y Trivy:** `pip-audit` detecta vulnerabilidades conocidas en dependencias de Python del entorno de ML; Trivy escanea imágenes Docker completas incluyendo el sistema operativo y todos los paquetes instalados.
- **SBOM (Software Bill of Materials) para ML:** documentación explícita de todos los componentes del sistema: modelo base con su versión y hash, datasets de entrenamiento con su fuente y versión, dependencias de Python con versiones exactas; permite auditoría y respuesta rápida ante vulnerabilidades descubiertas en cualquier componente.

Nota del Arquitecto: La supply chain de ML tiene un nivel de confianza implícita que sorprende a equipos acostumbrados a entornos más controlados. Descargamos modelos de 10GB de repositorios que cualquiera puede publicar, instalamos decenas de dependencias con `pip install`, y ejecutamos código de notebooks de Jupyter descargados de la web — todo esto en entornos que frecuentemente tienen acceso a credenciales de cloud, conexiones a bases de datos de producción, y sistemas de almacenamiento con datos de usuarios. El modelo correcto es tratarlo con la misma desconfianza que trataríamos código externo en cualquier otro contexto: verificar antes de ejecutar, aislar la ejecución, y auditar periódicamente las dependencias.

La supply chain security para ML es una laguna que muchos equipos no reconocen como riesgo hasta que ocurre un incidente. Los controles descritos en esta sección —safetensors, modelscan, pip-audit, verificación de hashes— son específicos de ML pero operacionalmente simples de implementar como parte del pipeline de descarga y actualización de modelos.

Módulo 9 – Capítulo 03 – Sección 06

Cierre: los ataques adversariales son una realidad en producción, no solo un problema de investigación

Durante varios años, los ataques adversariales contra sistemas de ML fueron considerados por la comunidad de práctica como curiosidades académicas con poca relevancia operacional: requerían acceso whitebox al modelo, condiciones de laboratorio controladas, y capacidades que estaban fuera del alcance de adversarios comunes. Esa percepción cambió de manera definitiva con una serie de demostraciones de viabilidad en producción que el equipo de AI Engineering moderno no puede ignorar: Carlini et al. extrayendo PII verbatim de GPT-2 y GPT-3.5 en producción, Zou et al. demostrando jailbreaks transferibles desde modelos open-source a GPT-4 sin acceso a sus internals, y múltiples reportes de Protect AI y otros investigadores documentando modelos maliciosos en Hugging Face Hub que ejecutaban código arbitrario al ser cargados.

El mapa actualizado del riesgo adversarial para el AI Engineer incluye cuatro categorías operacionalmente relevantes. La primera es la vulnerabilidad de los clasificadores-guardrail a perturbaciones adversariales: los modelos de moderación como LlamaGuard, Azure Content Safety y Perspective API son susceptibles a perturbaciones de caracteres y palabras que mantienen el significado para el LLM pero engañan al clasificador, lo que convierte la defensa basada en un único clasificador en una ilusión de seguridad. La segunda es la extracción del comportamiento propietario del modelo: cualquier modelo con comportamiento diferencial relevante —fine-tuneado sobre datos de dominio, adaptado con RLHF a un estilo específico— puede ser aproximado mediante queries sistemáticas a su API, y las defensas técnicas disponibles (rate limiting, watermarking, monitoreo de patrones) reducen pero no eliminan este riesgo. La tercera es la memorización de training data: el modelo puede revelar datos sensibles de su corpus de entrenamiento mediante técnicas de extracción documentadas, y la única defensa efectiva debe implementarse antes del entrenamiento (deduplicación, differential privacy), no después. La cuarta —y la más inmediatamente accionable— es la supply chain: modelos de Hugging Face en formato pickle pueden contener código malicioso

ejecutado al cargarse, y este riesgo puede ser mitigado completamente con `safetensors` y `modelscan`.

La defensa efectiva ante estos ataques comparte un patrón: los controles más efectivos se implementan antes del incidente, no como respuesta reactiva. La deduplicación del corpus de entrenamiento es mucho más fácil que el machine unlearning posterior. La verificación de la integridad del modelo antes de cargarlo es trivial comparada con el análisis forense de una intrusión. El uso de `safetensors` en lugar de pickle es un cambio de una línea de código que elimina una clase completa de vulnerabilidades. El threat model del sistema de IA debe incorporar estas amenazas desde el diseño inicial, no como adiciones tardías a un sistema ya en producción.

Conceptos clave del capítulo

- **Ataques adversariales en texto:** perturbaciones de caracteres (homoglifos, zero-width) y palabras (TextFooler, BERT-Attack) que engañan a clasificadores-guardrail mientras el LLM principal interpreta el texto correctamente; los clasificadores de moderación son el target operacional más relevante.
- **Transferibilidad:** jailbreaks y ejemplos adversariales generados contra modelos open-source transfieren a modelos propietarios; la opacidad del modelo no es una defensa efectiva ante surrogates de alta fidelidad disponibles públicamente.
- **Model extraction:** replicación del comportamiento propietario mediante queries sistemáticas a la API; mitigada con rate limiting agresivo, monitoreo de patrones, watermarking estadístico y la postura de que el pipeline de actualización es el activo más difícil de replicar.
- **Membership inference:** determinación estadística de si datos específicos fueron parte del corpus de entrenamiento; LLMs memorizan y reproducen datos del training set; mitigada con deduplicación antes del entrenamiento y DP-SGD durante el fine-tuning.
- **Supply chain security para ML:** riesgo pickle en modelos de Hugging Face, dependencias de Python vulnerables, datasets con data poisoning; mitigado con `safetensors`, `modelscan`, `pip-audit`, verificación de hashes y SBOM para ML.

"Adversarial examples are not bugs, they are features — and understanding that changes how we think about the security of machine learning systems." — Ian Goodfellow, investigador de DeepMind y creador de las Generative Adversarial Networks, sobre por qué la vulnerabilidad adversarial es una propiedad estructural de los modelos, no un defecto corregible con un patch.